# LAPORAN UJIAN TENGAH SEMESTER

# KONSTRUKSI DAN EVOLUSI PERANGKAT LUNAK

# SISTEM MANAJEMEN ASET LABORATORIUM DENGAN CI/CD DAN CONTAINERIZATION




**Disusun oleh:**

| | |
|---|---|
| **Nama** | : Hafidz Rizqullah Prasetya |
| **NIM** | : 24/535493/SV/24243 |
| **Kelas** | : PL5A1 |
| **Dosen Pengampu** | : Dr. Muhammad Fakhrurrifqi, S.Kom., M.Cs. |
| | Galih Malela Damaraji, S.Pd., M.Eng. |

**PROGRAM STUDI D-IV**

**TEKNOLOGI REKAYASA PERANGKAT LUNAK**

**DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA**

**SEKOLAH VOKASI**

**UNIVERSITAS GADJAH MADA**

**YOGYAKARTA**

**2026**

# Daftar Isi

**BAB I**
**PENDAHULUAN** . . . . . **1**
1.1 Latar Belakang . . . . . 1
1.2 Tujuan Ujian . . . . . 1
1.3 Batasan Masalah dan Lingkup Sistem . . . . . 2

**BAB II**
**DASAR TEORI** . . . . . **3**
2.1 Arsitektur RESTful API dan Autentikasi Token . . . . . 3
2.2 Framework Frontend Reaktif dan Klien HTTP . . . . . 3
2.3 Manajemen Repositori Git dan Proteksi Cabang . . . . . 4
2.4 Otomatisasi Alur Kerja dengan GitHub Actions . . . . . 4
2.5 Kontainerisasi dan Optimasi Dockerfile Multi-Stage . . . . . 4

**BAB III**
**HASIL DAN PEMBAHASAN** . . . . . **5**
3.1 Backend Laravel RESTful API (CPMK 1) . . . . . 5
3.1.1 Perancangan Skema Data dan Model . . . . . 5
3.1.2 Autentikasi Token Laravel Sanctum . . . . . 6
3.1.3 Implementasi Endpoint dan Status HTTP . . . . . 6
3.1.4 Koleksi Postman dan Pengujian Otomatis . . . . . 6
3.2 Frontend Vue.js dan Axios (CPMK 1) . . . . . 7
3.2.1 Konfigurasi Variabel Lingkungan . . . . . 8
3.2.2 Antarmuka Masuk Pengguna dan Manajemen Token . . . . . 8
3.2.3 Dasbor Metrik dan Pengelolaan Data Aset . . . . . 9
3.3 Manajemen Repositori GitHub dan CI Pipeline (CPMK 2) . . . . . 10
3.3.1 Struktur Repositori dan Strategi Percabangan . . . . . 10
3.3.2 Konfigurasi Branch Protection Rules . . . . . 11
3.3.3 Konfigurasi Workflow CI . . . . . 12
3.4 Continuous Deployment ke GHCR (CPMK 3) . . . . . 12
3.4.1 Pemisahan Perilaku Eksekusi Deployment . . . . . 12
3.4.2 Pengelolaan Kunci Rahasia dengan GitHub Secrets . . . . . 13
3.4.3 Pemberian Tag Berdasarkan Commit SHA dan Registri GHCR . . 13

3.5 Penyusunan Dockerfile Backend dan Frontend (CPMK 4 dan 5) . . . . . 15
3.5.1 Dockerfile Backend dengan Optimasi Layer Caching . . . . . 15
3.5.2 Dockerfile Frontend Multi-Stage Build . . . . . 16

**BAB IV**
**KESIMPULAN . . . . . 18**

# BAB I
# PENDAHULUAN

## 1.1 Latar Belakang

Pengembangan perangkat lunak modern menuntut integrasi yang solid antara arsitektur aplikasi dan siklus operasional pengembangan. Pendekatan tradisional yang memisahkan tahap pemrograman dengan tahap pengujian serta penerapan ke lingkungan produksi sering memicu inkonsistensi lingkungan, kegagalan integrasi, dan keterlambatan rilis. Konsep DevOps dan filosofi *agile* hadir untuk menjembatani kesenjangan tersebut melalui otomatisasi alur kerja, pengujian berkelanjutan, dan standarisasi lingkungan eksekusi berbasis kontainer.

Dalam konteks rekayasa sistem terdistribusi, pemisahan tanggung jawab antara sisi penyedia data (*backend*) dan antarmuka interaktif (*frontend*) merupakan standar industri. Sisi *backend* bertindak sebagai penyedia layanan RESTful API dengan protokol autentikasi berbasis token yang aman, sementara *frontend* berfokus pada pengalaman pengguna yang dinamis dan modular. Untuk menjamin keandalan sistem berskala produksi, seluruh kode sumber dikelola menggunakan sistem kendali versi Git dengan aturan proteksi cabang yang ketat, diverifikasi melalui saluran integrasi berkelanjutan (*Continuous Integration*), dan didistribusikan secara otomatis (*Continuous Deployment*) ke dalam registri kontainer seperti GitHub Container Registry (GHCR).

## 1.2 Tujuan Ujian

Tujuan pengerjaan Ujian Tengah Semester mata kuliah Konstruksi dan Evolusi Perangkat Lunak ini mencakup pencapaian kompetensi pada beberapa Capaian Pembelajaran Mata Kuliah (CPMK):

1. Menerapkan konsep dasar RESTful API, autentikasi berbasis token (Laravel Sanctum), dan antarmuka web modern (Vue.js 3) yang membaca konfigurasi dinamis (CPMK 1).

2. Menerapkan prinsip integrasi berkelanjutan (CI) dan manajemen repositori Git dengan aturan proteksi cabang dan verifikasi otomatis menggunakan GitHub Actions (CPMK

2).

3. Menerapkan penerapan berkelanjutan (CD) untuk membangun dan mempublikasikan citra kontainer ke GHCR dengan proteksi rahasia menggunakan GitHub Secrets (CPMK 3).

4. Membangun berkas konfigurasi *Dockerfile* untuk *backend* Laravel dengan optimasi *layer caching* dependensi Composer (CPMK 4).

5. Menerapkan teknik *multi-stage build* pada *Dockerfile frontend* Vue.js menggunakan peladen Nginx tanpa menyertakan berkas dependensi kerja (CPMK 5).

## 1.3 Batasan Masalah dan Lingkup Sistem

Lingkup pengerjaan ujian ini dibatasi pada perancangan dan implementasi Sistem Manajemen Inventaris Aset Laboratorium TRPL Sekolah Vokasi UGM. Sistem terdiri dari dua komponen utama:

1. **Backend:** Layanan berbasis Laravel dengan basis data SQLite, autentikasi token Sanctum, proteksi seluruh rute privat, respons JSON baku (status 200, 201, 401, 422), dan pengujian otomatis *feature test*.

2. **Frontend:** Aplikasi berbasis Vue.js 3 dan Axios dengan halaman masuk pengguna, dasbor metrik aset, tabel inventaris, modal formulir penambahan dan pengubahan, serta penghapusan data dengan konfirmasi.

# 1

# BAB II
# DASAR TEORI

## 2.1 Arsitektur RESTful API dan Autentikasi Token

REST (*Representational State Transfer*) merupakan gaya arsitektur komunikasi data yang memanfaatkan metode standar protokol HTTP, seperti GET, POST, PUT, dan DELETE. Setiap operasi mencerminkan manipulasi keadaan sumber daya (*resource*). Pada sistem RESTful yang stateless, peladen tidak menyimpan sesi pengguna di memori sisi server, melainkan memverifikasi setiap permintaan melalui token kriptografis pada tajuk HTTP *Authorization* [1].

Laravel Sanctum menyediakan mekanisme penerbitan token akses pribadi (*Personal Access Token*) yang ringan. Ketika pengguna berhasil melakukan autentikasi, peladen menghasilkan string acak terenkripsi yang disimpan dalam basis data. Klien menyimpan token ini dan melampirkannya dengan skema *Bearer Token* pada setiap permintaan ke endpoint terproteksi. Standarisasi respons HTTP menjadi indikator kepatuhan API, seperti status 201 untuk entitas baru, status 422 jika terjadi kegagalan validasi masukan, dan status 401 jika permintaan tidak menyertakan token yang sah [1].

## 2.2 Framework Frontend Reaktif dan Klien HTTP

Vue.js adalah kerangka kerja JavaScript progresif yang mengadopsi pola arsitektur berbasis komponen reaktif. Melalui Composition API, status aplikasi dipantau secara otomatis sehingga perubahan data langsung direfleksikan pada antarmuka pengguna tanpa memuat ulang peramban [2].

Komunikasi antara antarmuka Vue dan peladen API dijembatani oleh pustaka Axios. Axios mempermudah interceptor tajuk permintaan (*request interceptor*) untuk menyisipkan token secara konsisten, serta menangani kegagalan autentikasi (*response interceptor*) untuk membersihkan penyimpanan lokal saat token kedaluwarsa. Praktik rekayasa yang baik meng-

haruskan URL peladen dibaca dari variabel lingkungan (*environment variable*) melalui Vite, sehingga kode sumber tidak terikat pada alamat jaringan tertentu [3].

## 2.3 Manajemen Repositori Git dan Proteksi Cabang

Dalam alur kerja kolaboratif, cabang `main` diposisikan sebagai representasi kode produksi yang stabil, sedangkan cabang `dev` berfungsi sebagai wadah integrasi awal. Penambahan kapabilitas baru diisolasi pada cabang fitur (*feature branch*) dan hanya dapat digabungkan ke cabang utama melalui mekanisme *Pull Request* (PR) [5].

Fitur *Branch Protection Rules* di GitHub memungkinkan penegakan kebijakan keselamatan kode. Aturan ini melarang pengunggahan langsung (*direct push*) dan mewajibkan seluruh pengujian otomatis (*status checks*) berstatus lulus sebelum perubahan diizinkan untuk digabungkan. Pendekatan ini meminimalkan risiko regresi dan memastikan integritas basis kode tetap terjaga [5].

## 2.4 Otomatisasi Alur Kerja dengan GitHub Actions

GitHub Actions adalah platform integrasi dan penerapan berkelanjutan terkelola yang mengeksekusi serangkaian pekerjaan (*jobs*) berdasarkan kejadian tertentu pada repositori (*triggers*). Alur kerja didefinisikan dalam format YAML di direktori `.github/workflows/` [4].

Pemisahan pekerjaan antara pengujian sisi server (*backend test*) dan kompilasi sisi klien (*frontend build*) memungkinkan eksekusi secara paralel guna mempercepat umpan balik. Pada tahap lanjutan, pekerjaan penerapan (*deploy job*) diatur agar hanya aktif ketika commit berhasil digabungkan ke cabang `main`, sedangkan pada cabang lain alur kerja otomatis berhenti setelah pengujian selesai [4].

## 2.5 Kontainerisasi dan Optimasi Dockerfile Multi-Stage

Docker mengemas aplikasi beserta pustaka dan lingkungan dependensinya ke dalam satu unit mandiri yang disebut kontainer. Mekanisme ini menjamin bahwa aplikasi berjalan dengan perilaku yang identik di mesin pengembang, peladen CI, maupun peladen produksi [7].

Penyusunan berkas *Dockerfile* yang optimal memanfaatkan urutan lapisan instruksi (*layer caching*). Dengan menyalin manifes dependensi (seperti `composer.json` dan `package.json`) sebelum kode sumber aplikasi, proses unduhan pustaka tidak perlu diulang jika kode sumber mengalami revisi. Pada aplikasi sisi klien, teknik *multi-stage build* memisahkan lingkungan kompilasi berbasis Node.js dengan lingkungan eksekusi berbasis web server Nginx berukuran ringkas, sehingga berkas sementara dan pustaka pengembang tidak ikut terbawa ke citra akhir [7, 8].

# 2

# BAB III
# HASIL DAN PEMBAHASAN

## 3.1 Backend Laravel RESTful API (CPMK 1)

Pengerjaan soal nomor 1 difokuskan pada pembangunan layanan RESTful API menggunakan Laravel dengan tema Sistem Manajemen Aset Laboratorium. Sistem mengelola data inventaris perangkat keras dan instrumen yang digunakan dalam kegiatan praktikum di lingkungan kampus.

### 3.1.1 Perancangan Skema Data dan Model

Sumber daya utama yang dikelola adalah `Asset`. Tabel `assets` dirancang untuk menampung atribut kode inventaris unik, nama barang, kategori, status ketersediaan, nilai perolehan, dan lokasi laboratorium. Struktur migrasi basis data ditunjukkan pada Listing 3.1.

```
Schema::create('assets', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('code')->unique();
    $table->string('category');
    $table->string('status')->default('tersedia');
    $table->decimal('price', 15, 2);
    $table->string('location')->nullable();
    $table->timestamps();
});
```

Listing 3.1: Migrasi tabel assets pada backend Laravel.

Model `Asset` dilengkapi atribut pengisian massal (*fillable*) untuk mencegah kerentanan *mass assignment*, serta *casting* tipe data desimal menjadi nilai numerik float pada respons JSON.

### 3.1.2 Autentikasi Token Laravel Sanctum

Autentikasi dikelola oleh `AuthController`. Pengguna melakukan autentikasi melalui rute publik `POST /api/login`. Jika alamat email dan kata sandi valid, peladen menghasilkan token teks biasa melalui metode `createToken()` dari trait `HasApiTokens`. Token tersebut dikirimkan kembali ke pengguna bersama informasi profil dasar.

Seluruh rute sumber daya dilindungi oleh middleware `auth:sanctum`. Jika pengguna mengirimkan permintaan tanpa menyertakan token atau menggunakan token yang sudah tidak berlaku, penanganan pengecualian pada `bootstrap/app.php` secara otomatis menangkap `AuthenticationException` dan mengembalikan respons JSON dengan status **401 Unauthorized**. Rute `POST /api/logout` mencabut token aktif pengguna sehingga token tersebut tidak dapat digunakan kembali.

### 3.1.3 Implementasi Endpoint dan Status HTTP

Operasi manipulasi data pada `AssetController` mengembalikan kode status HTTP standar sesuai ketentuan soal:

1. `200 OK`: Dikembalikan saat pengambilan daftar aset (`GET /api/assets`), detail satu aset (`GET /api/assets/{id}`), pembaruan data (`PUT /api/assets/{id}`), dan penghapusan data (`DELETE /api/assets/{id}`).

2. `201 Created`: Dikembalikan secara eksplisit saat penambahan entitas aset baru berhasil disimpan (`POST /api/assets`).

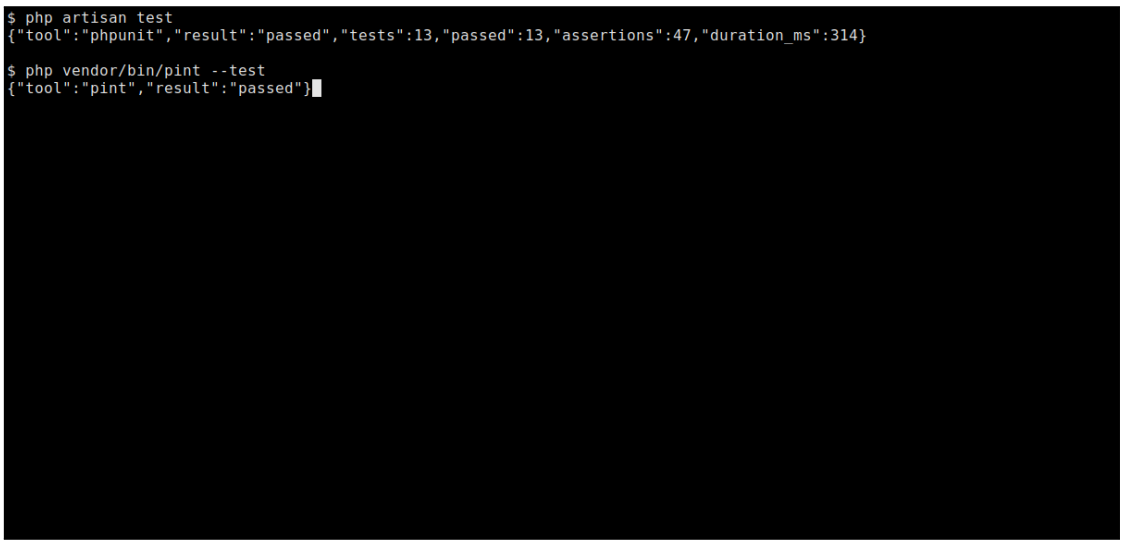
3. `422 Unprocessable Content`: Dikembalikan oleh validator Laravel apabila masukan data tidak memenuhi aturan validasi (misalnya nama kosong, format harga bernilai negatif, atau kode aset sudah terdaftar).

4. `401 Unauthorized`: Dikembalikan saat rute privat diakses tanpa menyertakan Bearer Token yang sah.

### 3.1.4 Koleksi Postman dan Pengujian Otomatis

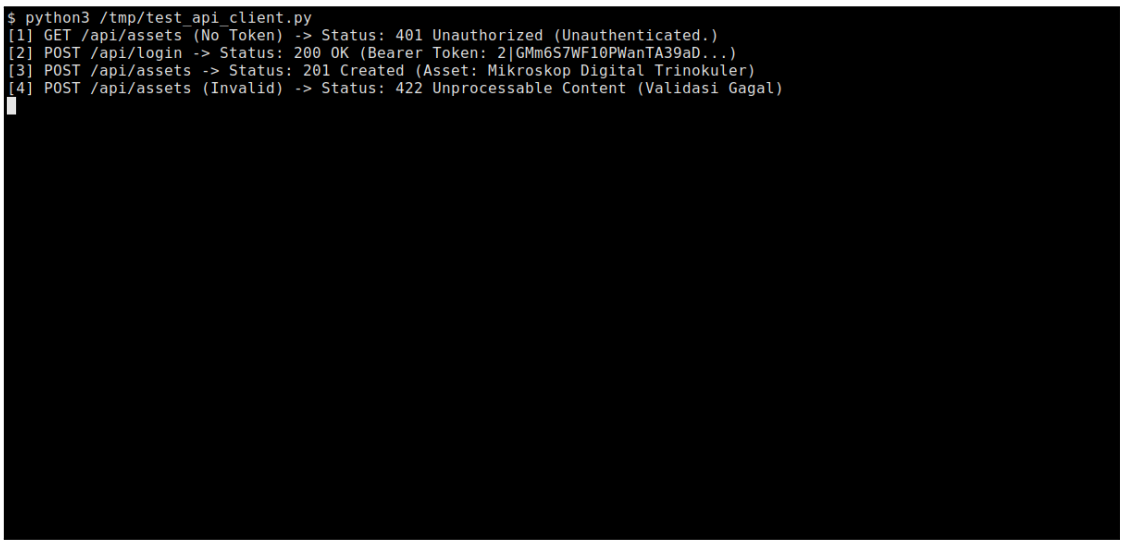
Seluruh endpoint diuji menggunakan koleksi Postman yang telah diekspor dan disimpan pada berkas `backend/postman/uts_kepl_asset_api.postman_collection.json`. Koleksi ini mencakup skrip uji otomatis yang menyimpan token sesi ke variabel koleksi secara instan setelah panggilan login berhasil.

Selain pengujian manual, keandalan sistem diverifikasi menggunakan pengujian unit dan fitur terotomatisasi (*PHPUnit Feature Tests*) pada `tests/Feature/AuthApiTest.php` dan `tests/Feature/AssetApiTest.php`. Seluruh 13 pengujian dengan 47 penegasan (*assertions*) berhasil diselesaikan dengan status lulus, sebagaimana ditunjukkan pada Gambar 1.




Gambar 1: Hasil eksekusi pengujian otomatis `php artisan test` dan pemeriksaan standar format kode `pint`.

Verifikasi kepatuhan kode status HTTP secara langsung melalui terminal ditampilkan pada Gambar 2, yang membuktikan bahwa respons kode 200, 201, 401, dan 422 bekerja secara konsisten.




Gambar 2: Verifikasi status code HTTP 401, 200, 201, dan 422 melalui pemanggilan klien API terminal.

## 3.2 Frontend Vue.js dan Axios (CPMK 1)

Pengerjaan soal nomor 2 mencakup pembangunan antarmuka pengguna berbasis Vue.js 3 dengan bundler Vite dan pustaka HTTP Axios.

### 3.2.1 Konfigurasi Variabel Lingkungan

Untuk memenuhi ketentuan soal 2.c, alamat basis peladen API tidak ditulis mati di dalam kode komponen. Alamat dibaca dari berkas konfigurasi lingkungan `.env` melalui variabel `VITE_API_BASE_URL`, seperti ditunjukkan pada Listing 3.2.

```
import axios from 'axios';

const apiBaseUrl = import.meta.env.VITE_API_BASE_URL || 'http://localhost:8000/api';

const apiClient = axios.create({
  baseURL: apiBaseUrl,
  headers: {
    'Accept': 'application/json',
    'Content-Type': 'application/json',
  },
});

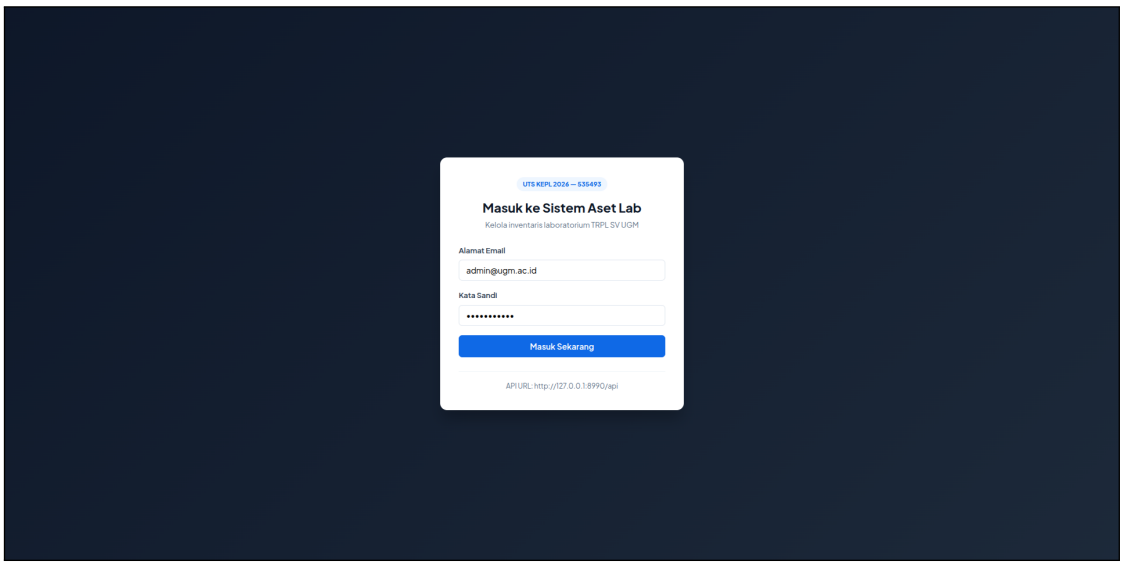
apiClient.interceptors.request.use((config) => {
  const token = localStorage.getItem('auth_token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

export default apiClient;
```

Listing 3.2: Konfigurasi klien Axios dengan pembacaan variabel lingkungan.

### 3.2.2 Antarmuka Masuk Pengguna dan Manajemen Token

Ketika pengguna mengakses aplikasi untuk pertama kali, sistem menampilkan formulir masuk. Gambar 3 memperlihatkan antarmuka halaman login yang menyertakan informasi alamat endpoint aktif yang dibaca dari variabel lingkungan.

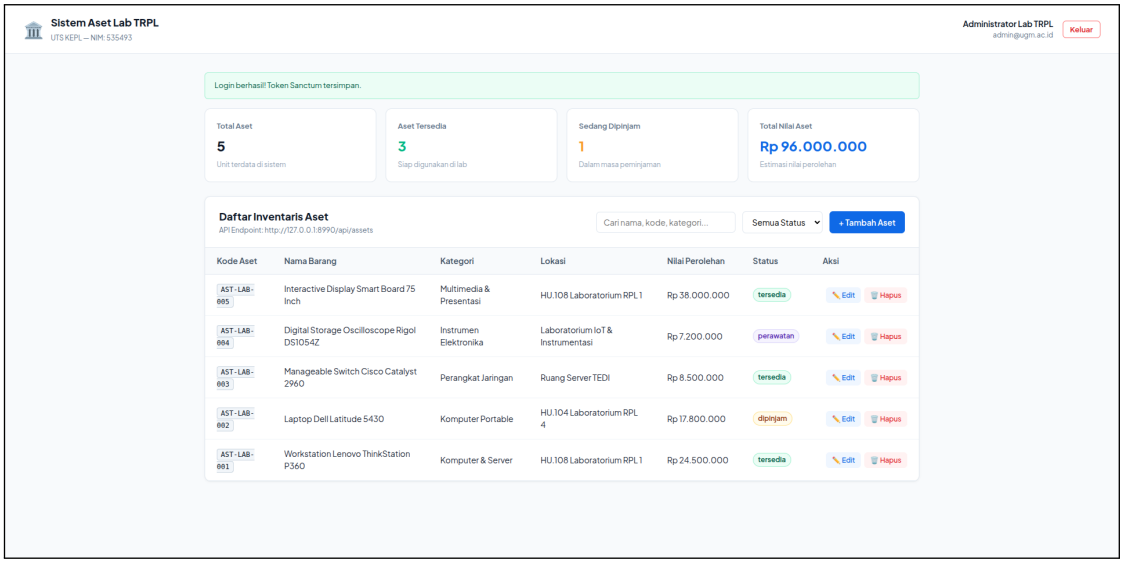



Gambar 3: Halaman login Vue.js dengan pembacaan konfigurasi variabel lingkungan API.

Saat formulir dikirimkan, fungsi `handleLogin()` mengirim permintaan autentikasi ke backend. Token yang diterima disimpan ke dalam `localStorage` dengan kunci `auth_token`. Berkat *interceptor* Axios, token ini secara otomatis disertakan pada setiap permintaan HTTP berikutnya.

### 3.2.3 Dasbor Metrik dan Pengelolaan Data Aset

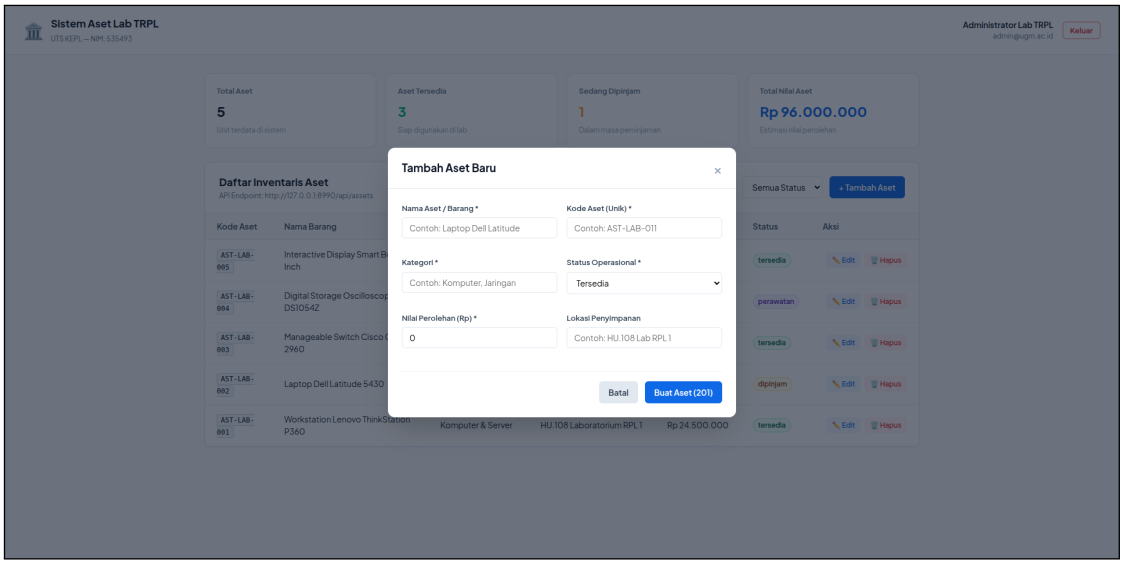
Setelah masuk, pengguna diarahkan ke antarmuka dasbor inventaris. Dasbor menampilkan ringkasan statistik (total aset, unit tersedia, unit dipinjam, dan total valuasi perolehan), fitur pencarian teks, penyaringan berdasarkan status, serta tabel inventaris lengkap. Tampilan dasbor disajikan pada Gambar 4.




Gambar 4: Dasbor manajemen inventaris aset laboratorium pada antarmuka Vue.js.

Penambahan aset baru dilakukan melalui jendela modal interaktif. Modal ini melakukan

validasi masukan pada sisi klien sebelum mengirimkan data ke rute `POST /api/assets`. Apabila peladen mengembalikan kode 201 Created, daftar data pada tabel langsung diperbarui secara reaktif tanpa memuat ulang peramban. Tampilan modal penambahan aset ditunjukkan pada Gambar 5.




Gambar 5: Modal interaktif penambahan aset laboratorium baru pada antarmuka frontend.

## 3.3 Manajemen Repositori GitHub dan CI Pipeline (CPMK 2)

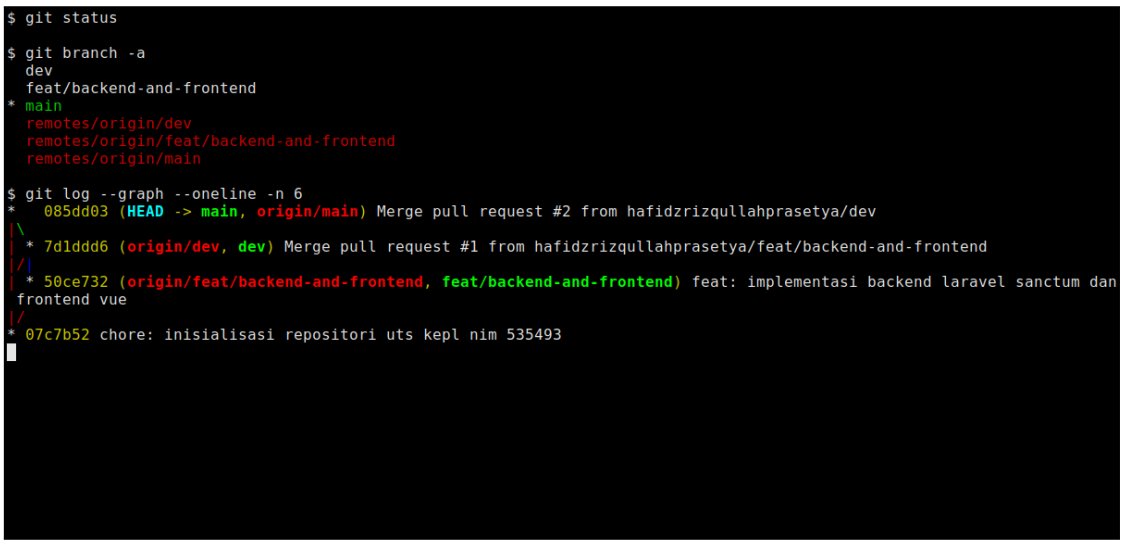
Soal nomor 3 mewajibkan pengelolaan seluruh kode sistem pada repositori publik bernama `uts-p1-535493` di akun GitHub pribadi (`https://github.com/hafidzrizqullahprasetya/uts-p1-535493`), memuat folder `backend/` dan `frontend/`, serta menerapkan aturan cabang terlindungi.

### 3.3.1 Struktur Repositori dan Strategi Percabangan

Repositori diatur dengan tiga tingkatan cabang:

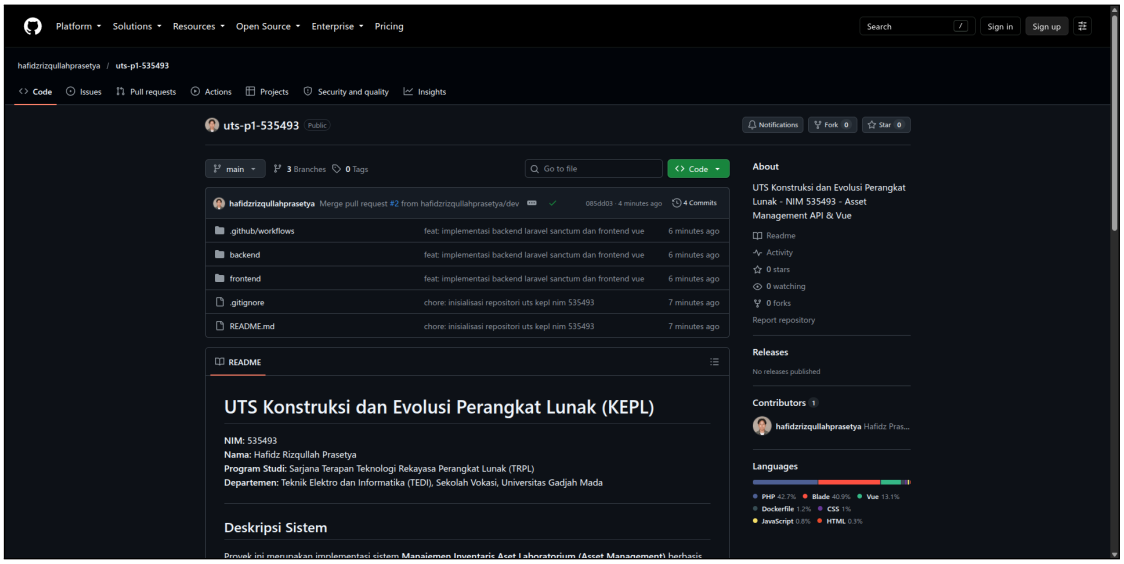
1. `main`: Cabang produksi utama yang terlindungi secara ketat.
2. `dev`: Cabang integrasi pengembangan pra-produksi.
3. `feat/backend-and-frontend`: Cabang fitur yang menampung pengembangan awal kode backend, frontend, konfigurasi Docker, dan alur CI/CD.

Seluruh perubahan kode dari cabang fitur diintegrasikan ke cabang `dev` melalui Pull Request nomor 1. Setelah diverifikasi lulus oleh pipa CI, cabang `dev` digabungkan ke cabang `main` melalui Pull Request nomor 2. Riwayat komit dan topologi pohon percabangan pada repositori ditampilkan pada Gambar 6.




Gambar 6: Pemeriksaan status cabang Git lokal dan riwayat graf penggabungan Pull Request.

Tampilan beranda repositori publik di GitHub disajikan pada Gambar 7, yang memperlihatkan struktur direktori `backend/` dan `frontend/` yang bersih.



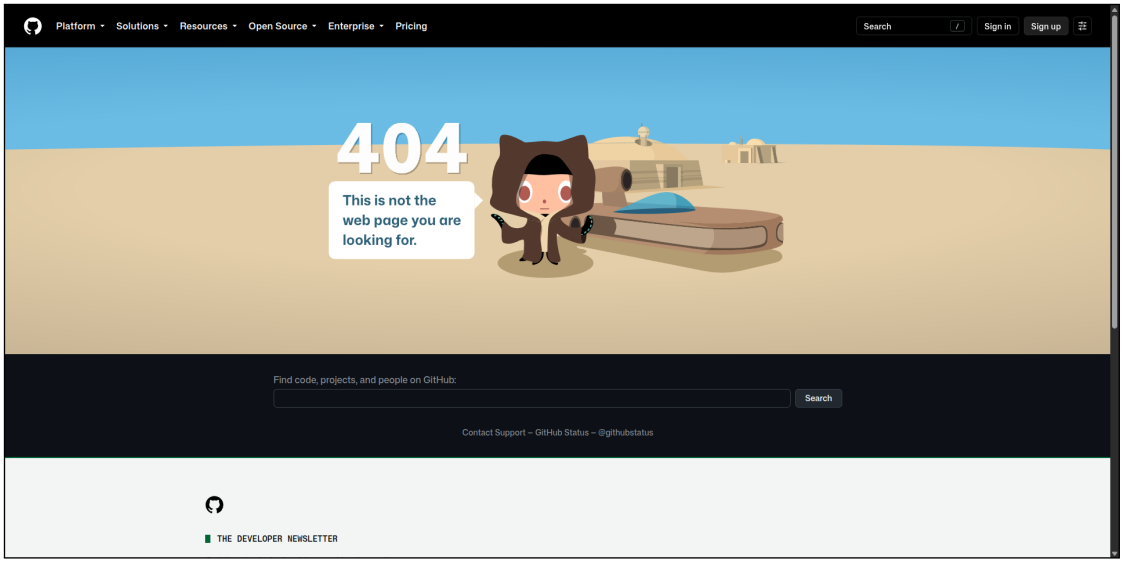

Gambar 7: Tampilan repositori publik `uts-p1-535493` pada platform GitHub.

### 3.3.2 Konfigurasi Branch Protection Rules

Sesuai instruksi soal, cabang `main` dan `dev` diproteksi untuk mencegah pengunggahan langsung (*direct push*). Pengaturan proteksi cabang mewajibkan:

1. Integrasi hanya dapat dilakukan melalui mekanisme Pull Request.
2. Pemeriksaan status (*status checks*) harus lulus sebelum penggabungan diizinkan, yaitu job `Backend Test` dan `Frontend Build`.
3. Opsi cabang harus mutakhir (*strict check: true*) sebelum digabungkan.

Bukti konfigurasi Branch Protection Rules pada antarmuka pengaturan GitHub ditunjukkan pada Gambar 8.




Gambar 8: Pengaturan Branch Protection Rules untuk cabang `main` dan `dev` pada repositori GitHub.

### 3.3.3 Konfigurasi Workflow CI

Alur kerja integrasi berkelanjutan didefinisikan pada berkas `.github/workflows/ci.yml`. Berkas ini mengonfigurasi dua pekerjaan pengujian yang berjalan secara independen:

1. **Backend Test:** Mengatur lingkungan PHP 8.4, memasang dependensi Composer dari cache, menyiapkan basis data SQLite lokal, menjalankan migrasi, dan mengeksekusi `php artisan test`.

2. **Frontend Build:** Mengatur lingkungan Node.js 20, memasang pustaka secara bersih menggunakan `npm ci`, dan memverifikasi kompilasi aset produksi melalui `npm run build`.

## 3.4 Continuous Deployment ke GHCR (CPMK 3)

Soal nomor 4 mewajibkan penambahan pekerjaan penerapan (*deploy job*) pada alur kerja GitHub Actions untuk mempublikasikan kontainer ke GitHub Container Registry (`ghcr.io`).

### 3.4.1 Pemisahan Perilaku Eksekusi Deployment

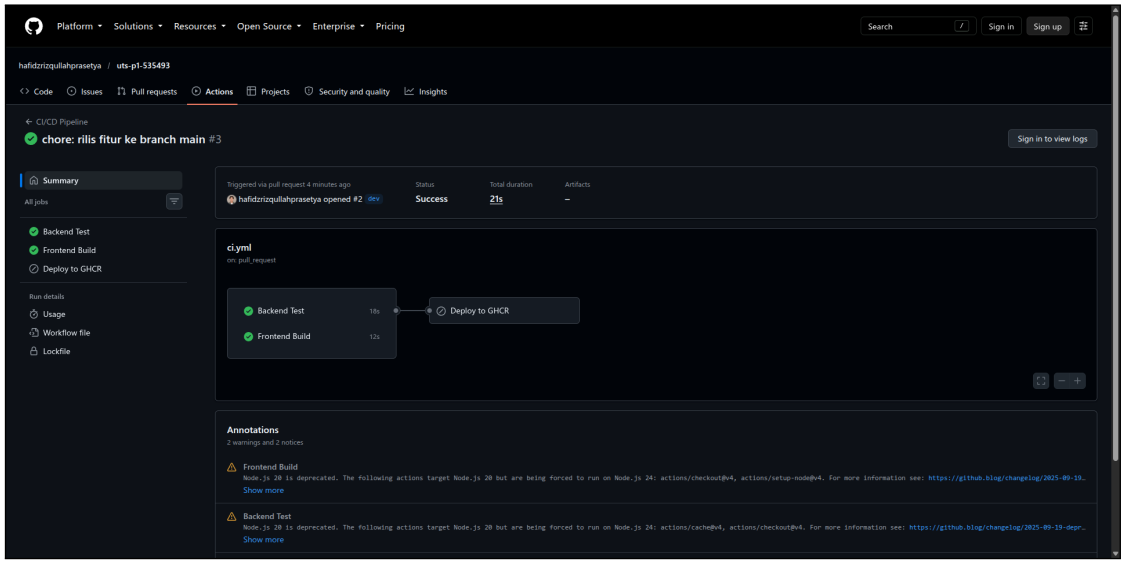
Sesuai butir soal 4.a, pekerjaan penerapan hanya boleh dijalankan saat terjadi peristiwa pengunggahan (*push*) ke cabang `main`. Pada pengujian Pull Request atau pengunggahan ke

cabang `dev`, pekerjaan `deploy` dilewati secara otomatis. Kondisi logika ini didefinisikan pada deklarasi pekerjaan:

```
deploy:
  name: Deploy to GHCR
  needs: [backend-test, frontend-build]
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
```

Listing 3.3: Potongan kondisi eksekusi job deploy pada ci.yml.

Bukti eksekusi alur kerja pada Pull Request cabang `dev` ditunjukkan pada Gambar 9, di mana pekerjaan `Backend Test` dan `Frontend Build` berhasil diselesaikan dengan warna hijau, sedangkan pekerjaan `Deploy to GHCR` dilewati (*skipped*).




Gambar 9: Riwayat alur kerja GitHub Actions pada Pull Request yang melewati tahap deployment.

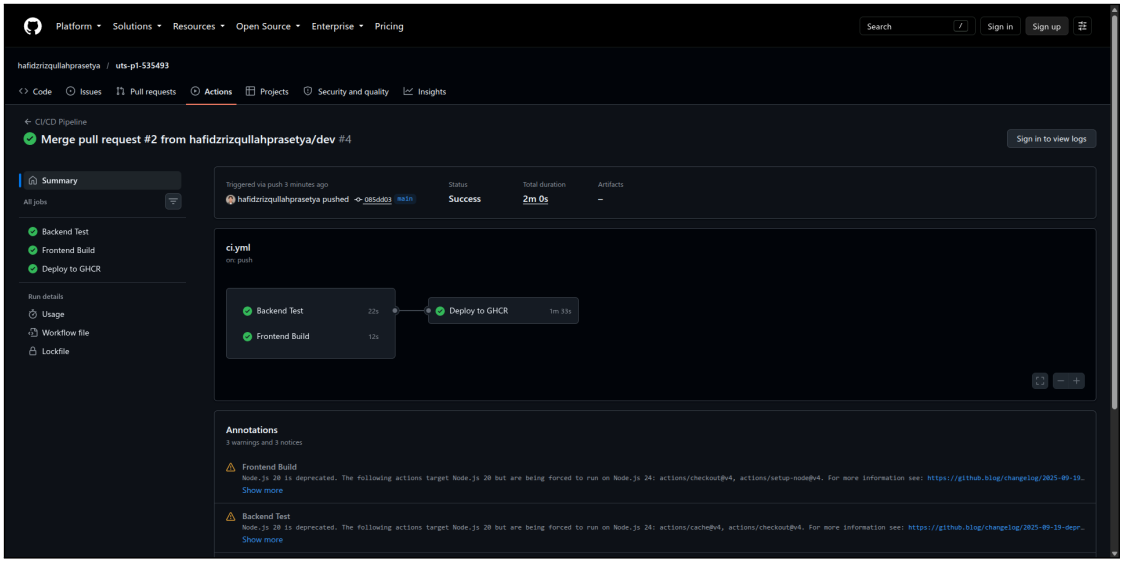
### 3.4.2 Pengelolaan Kunci Rahasia dengan GitHub Secrets

Sesuai butir soal 4.b, nilai konfigurasi sensitif seperti `APP_KEY` dan `DB_PASSWORD` disimpan pada menu *Settings* → *Secrets and variables* → *Actions* di repositori GitHub. Variabel ini disuntikkan ke dalam proses pembangunan citra kontainer sebagai *build-args* tanpa pernah ditulis secara terbuka di dalam repositori publik.

### 3.4.3 Pemberian Tag Berdasarkan Commit SHA dan Registri GHCR

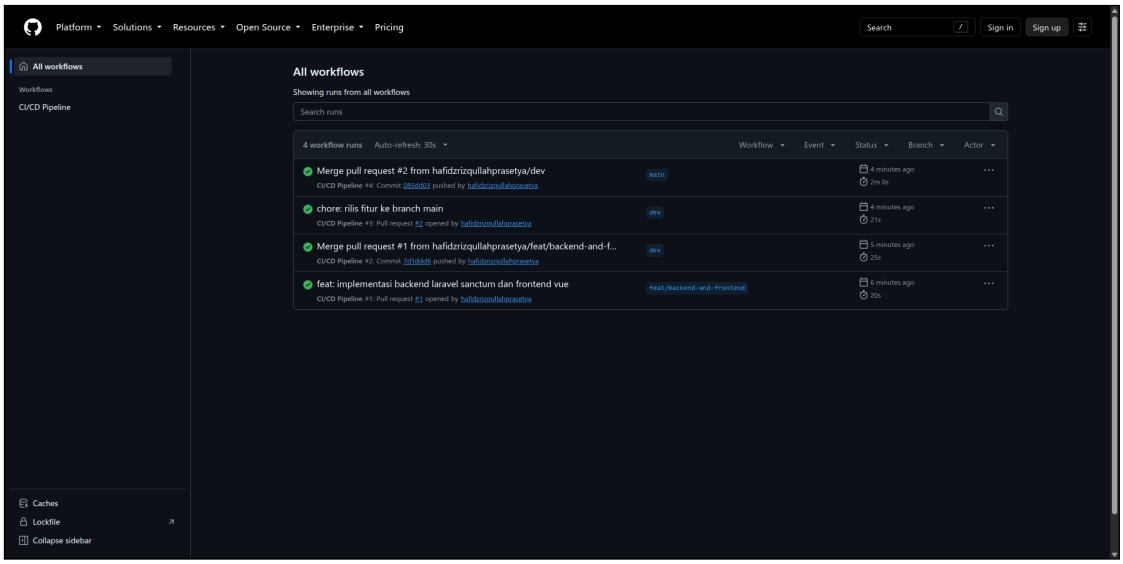
Pekerjaan penerapan melakukan autentikasi ke registri `ghcr.io` menggunakan kredensial `GITHUB_TOKEN` yang memiliki izin `packages: write`. Kontainer backend dan frontend dibangun dan diunggah dengan tag yang merujuk pada nomor commit penuh (`${{ github.sha }}`), bukan tag `latest`. Hal ini menjamin keterlacakan (*traceability*) antara berkas artefak kontainer dan kode sumber yang menyusunnya.

Gambar 10 memperlihatkan eksekusi alur kerja pada cabang `main` yang berhasil menyelesaikan ketiga pekerjaan secara berurutan dengan status hijau sempurna.



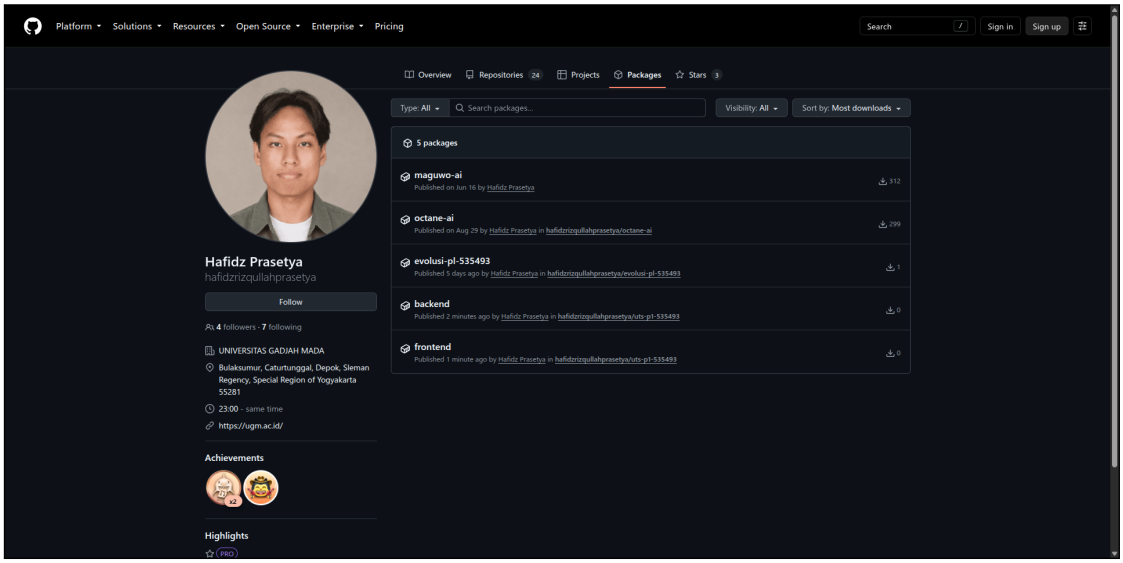

Gambar 10: Eksekusi alur kerja CI/CD pada cabang `main` dengan tahap deploy yang berhasil diselesaikan.

Seluruh riwayat alur kerja terangkum pada tab Actions repositori, sebagaimana disajikan pada Gambar 11.




Gambar 11: Ringkasan riwayat eksekusi pipa CI/CD pada tab Actions repositori GitHub.

Hasil publikasi kedua citra kontainer pada GitHub Container Registry ditampilkan pada Gambar 12. Kedua paket (`uts-p1-535493/backend` dan `uts-p1-535493/frontend`) terdaftar di bawah namespace repositori dan siap diunduh untuk kebutuhan orkestrasi peladen.




Gambar 12: Halaman registri paket GitHub Container Registry memuat citra backend dan frontend.

## 3.5 Penyusunan Dockerfile Backend dan Frontend (CPMK 4 dan 5)

Soal nomor 5 menuntut pembuatan berkas *Dockerfile* untuk backend dan frontend dengan standar optimasi kontainerisasi industri.

### 3.5.1 Dockerfile Backend dengan Optimasi Layer Caching

Berkas `backend/Dockerfile` menggunakan citra dasar resmi `php:8.4-cli-alpine` yang eksplisit dan tidak menggunakan tag `latest`. Untuk mengoptimalkan kinerja *cache* lapisan Docker, berkas manifes dependensi `composer.json` dan `composer.lock` disalin terlebih dahulu, diikuti eksekusi instalasi pustaka Composer tanpa skrip pengembang. Kode sumber aplikasi baru disalin pada lapisan berikutnya, seperti disajikan pada Listing 3.4.

```
FROM php:8.4-cli-alpine

WORKDIR /var/www/html

RUN apk add --no-cache \
    curl git unzip libzip-dev sqlite-dev \
    libpng-dev oniguruma-dev libxml2-dev linux-headers \
    && docker-php-ext-install -j$(nproc) \
    pdo pdo_sqlite mbstring xml zip bcmath

COPY --from=composer:2.8 /usr/bin/composer /usr/bin/composer

# Optimasi Layer Cache: Salin manifes dependensi terlebih dahulu
```

```
COPY composer.json composer.lock ./
RUN composer install --no-dev --no-scripts --no-autoloader --prefer-dist

# Salin sisa kode sumber aplikasi
COPY . .

RUN composer dump-autoload --optimize \
    && cp -n .env.example .env || true \
    && touch database/database.sqlite \
    && php artisan key:generate --force \
    && chown -R www-data:www-data storage bootstrap/cache database

EXPOSE 8000
CMD ["php", "artisan", "serve", "--host=0.0.0.0", "--port=8000"]
```

Listing 3.4: Berkas backend/Dockerfile dengan optimasi Composer layer caching.

### 3.5.2 Dockerfile Frontend Multi-Stage Build

Berkas `frontend/Dockerfile` mengimplementasikan teknik *multi-stage build*. Pada tahap pertama (`builder`), citra menggunakan `node:20-alpine` untuk memasang dependensi dan membangun aset web produksi melalui `npm run build`. Pada tahap kedua, citra produksi menggunakan peladen `nginx:1.27-alpine`. Berkas hasil build dari direktori `/app/dist` disalin ke direktori publik Nginx (`/usr/share/nginx/html`), sedangkan direktori `node_modules` ditinggalkan pada tahap pertama. Pendekatan ini menghasilkan citra kontainer akhir yang sangat ramping dan aman, sebagaimana dicantumkan pada Listing 3.5.

```
# Stage 1: Lingkungan build Node.js
FROM node:20-alpine AS builder

WORKDIR /app
COPY package*.json ./
RUN npm ci

COPY . .
RUN npm run build

# Stage 2: Lingkungan produksi Nginx
FROM nginx:1.27-alpine

RUN rm -rf /usr/share/nginx/html/*
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=builder /app/dist /usr/share/nginx/html

EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Listing 3.5: Berkas frontend/Dockerfile menggunakan teknik multi-stage build.

# 3

# BAB IV
# KESIMPULAN

Berdasarkan seluruh rangkaian perancangan, implementasi, dan pengujian yang telah dilaksanakan pada pengerjaan Ujian Tengah Semester mata kuliah Konstruksi dan Evolusi Perangkat Lunak, dapat ditarik beberapa kesimpulan sebagai berikut:

1. Layanan backend berbasis Laravel RESTful API berhasil dibangun dengan tema Sistem Manajemen Aset Laboratorium. Autentikasi berbasis token Laravel Sanctum berhasil memproteksi seluruh rute privat, dengan implementasi kode status HTTP standar (201 Created, 422 Unprocessable Content, 401 Unauthorized, dan 200 OK) yang terverifikasi melalui 13 pengujian otomatis berstatus lulus dan koleksi Postman yang lengkap (CPMK 1).

2. Antarmuka frontend berbasis Vue.js 3 dan Axios berhasil mengintegrasikan alur login, persistensi token pada penyimpanan lokal, dan operasi CRUD aset secara reaktif. URL API berhasil dipisahkan dari logika komponen melalui konfigurasi variabel lingkungan `VITE_API_BASE_URL` (CPMK 1).

3. Repositori sistem telah dikelola secara terstruktur di GitHub pada repositori publik `uts-p1-535493` dengan strategi cabang `main`, `dev`, dan cabang fitur. Aturan *Branch Protection Rules* berhasil ditegakkan untuk mewajibkan alur Pull Request dan kelulusan pengujian CI sebelum penggabungan kode diizinkan (CPMK 2).

4. Jalur penerapan berkelanjutan (*Continuous Deployment*) berhasil dikonfigurasi pada alur kerja GitHub Actions. Pekerjaan deploy berhasil dibatasi hanya untuk kejadian push ke cabang `main`, memanfaatkan GitHub Secrets untuk variabel sensitif, dan mempublikasikan citra kontainer ke GitHub Container Registry (`ghcr.io`) dengan tag nomor commit SHA (CPMK 3).

5. Kontainerisasi sistem berhasil dioptimalkan melalui penyusunan *Dockerfile backend* yang memanfaatkan *layer caching* dependensi Composer, serta *Dockerfile frontend*

yang menerapkan arsitektur *multi-stage build* berbasis Nginx tanpa menyertakan direktori `node_modules` pada citra akhir (CPMK 4 dan CPMK 5).

# Daftar Pustaka


[1] Laravel LLC, "Laravel Sanctum - Light-weight authentication for SPAs and token APIs," 2026. [Daring]. Tersedia: https://laravel.com/framework/docs/sanctum. [Diakses: 16-Okt-2026].

[2] Vue.js Core Team, "Introduction to Vue.js - The Progressive JavaScript Framework," 2026. [Daring]. Tersedia: https://vuejs.org/guide/introduction.html. [Diakses: 16-Okt-2026].

[3] Axios Contributors, "Axios HTTP Client - Getting Started," 2026. [Daring]. Tersedia: https://axios.rest/pages/getting-started/first-steps. [Diakses: 16-Okt-2026].

[4] GitHub Inc., "Understanding GitHub Actions - Workflow Automation and CI/CD," 2026. [Daring]. Tersedia: https://docs.github.com/en/actions/get-started/understand-github-actions. [Diakses: 16-Okt-2026].

[5] GitHub Inc., "About Protected Branches and Pull Request Gates," 2026. [Daring]. Tersedia: https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches. [Diakses: 16-Okt-2026].

[6] GitHub Inc., "Working with the GitHub Container Registry (GHCR)," 2026. [Daring]. Tersedia: https://docs.github.com/en/packages/working-with-a-github-packages-registry/working-with-the-container-registry. [Diakses: 16-Okt-2026].

[7] Docker Inc., "Multi-stage Builds - Optimize Container Image Sizes and Security," 2026. [Daring]. Tersedia: https://docs.docker.com/build/building/multi-stage/. [Diakses: 16-Okt-2026].

[8] Nginx Inc., "Nginx Beginner's Guide - Web Server Configuration for Static Applications," 2026. [Daring]. Tersedia: https://nginx.org/en/docs/beginners_guide.html. [Diakses: 16-Okt-2026].